Fewer, Better Tokens

A Hardware-Aware Approach to Token Efficiency in an Operational Knowledge Assistant

Salina Giri · Applied Computer Science, UC Leuven-Limburg (UCLL), Belgium · November 2026

Technical exercise, internship 2026-INT-298 · Code and data:
https://github.com/SalinaGiri/token-efficient-rag-agent

Abstract. LLM inference runs in two phases with different hardware behaviour: a parallel, compute-bound *prefill* over input tokens and a sequential, memory-bandwidth-bound *decode* that generates output tokens [1, 2]. This paper treats token efficiency as a hardware-aware design problem: it reviews five optimisation levers, selects one per phase (adaptive retrieval for input, output budgeting for output) and evaluates them in a five-variant ablation on 40 questions, seven of them unanswerable. Adaptive retrieval cut input tokens by 91% and mean latency from 26.1 s to 4.0 s without loss of correctness, and raised abstention accuracy from 71% to 100%. Output budgeting cut output tokens by 56% but cost 18 points of correctness. On the laptop GPU used, one output token cost only about 8 input tokens of latency, so the input side dominated.

1. Introduction

As a knowledge base grows, every request to the LLM carries more tokens, raising cost, latency and energy, and agents pay this at every step; yet careless cuts remove evidence and citations. This study asks: **(RQ1)** how far can input and output tokens be reduced while correctness stays within a pre-defined margin; **(RQ2)** do input and output tokens have different latency costs; and **(RQ3)** does adaptive retrieval improve correct abstention on unanswerable questions?

2. Literature Review: The Optimisation Space

More context is not always better: models use evidence in the middle of long inputs less reliably [4] and are distracted by irrelevant passages [5], so the aim is the smallest set of high-signal tokens [6]. On the hardware side, Splitwise [1] and DistServe [2] separate the compute-bound prefill from the memory-bound decode, and PagedAttention [3] shows that the KV cache created by long inputs limits how many users an accelerator can serve. These works optimise the serving system; this study applies the same phase distinction to the *application*, which decides how many tokens of each kind are sent. Building on the RAG survey in [13], Table 1 organises five levers.

Table 1. Five levers for reducing token usage. Prompt caching is excluded on purpose: it saves recomputation, not tokens, and complements every lever.

Lever	How it works	Reduces	Expected benefit	Risk
T1 Adaptive retrieval	Rerank candidates [15]; keep only passages above a threshold τ, so the number k varies and may be 0 [7, 8].	Input (prefill, KV cache)	Shrinks the largest input source; k = 0 gives a token-free “not found”.	Strict τ drops needed evidence.
T2 Context compression	Keep query-relevant sentences or summarise passages [9, 10].	Input	As little as 6% of tokens with minimal loss [9].	Summaries break citation traceability.
T3 History compaction	Recent turns verbatim, older turns summarised, used tool outputs dropped [6].	Input	Stops history growth; savings multiply over agent steps.	Early user constraints lost.
T4 Output budgeting	Short, cited answer contract; concise reasoning [11]; token cap.	Output (decode)	Targets the slowest, sequential phase.	Too terse; truncation.
T5 Model cascade	Small model first; escalate when a confidence check fails [12].	Cost per token	Large cost savings reported [12].	Failed attempts paid twice.

3. Methodology

Selection and rationale. T1 and T4 cover *one inference phase each*, which makes RQ2 testable; they need no training and keep source text verbatim, so every citation points to a real passage. T2 risks traceability, T3 needs multi-turn data and T5 a calibrated confidence signal: the next iteration.

Data and materials. Knowledge base: the Kubernetes troubleshooting documentation (snapshot of 10 November 2026: 111 sections, 232 passages of ~250 tokens, about 36,000 tokens in total). Test

set: 40 questions written by hand with reference answers and gold passages checked manually against the corpus (25 single-fact, 8 two-passage, 7 unanswerable). Model: Qwen2.5-3B-Instruct (Q4_K_M) served locally by Ollama on an Apple M4 MacBook Air, temperature 0; retrieval with all-MiniLM-L6-v2 [16] and a MiniLM cross-encoder [15]. All 1,120 runs (520 in the τ sweep, 600 in the main experiment: 40 questions × 5 variants × 3 repeats) were executed on this machine, 975 of them with an Ollama call.

Procedure. T1 retrieves 20 candidates, reranks them and keeps up to four with score ≥ τ; if none qualifies, the agent answers "not found" *without calling the LLM*. τ was swept over six values and set to 0 (Figure 2b). T4 enforces an answer contract (answer first, at most three sentences, passage IDs as citations) with a 150-token cap. Five variants form an ablation: **V0** baseline (top-10 whole documents, free-form answer), **V1** T1 only, **V2** T4 only, **V3** T1 + T4, and **VO** an oracle given only the gold passages (with T4), which bounds the possible saving.

Analysis. Efficiency: tokens, time to first token (TTFT) and end-to-end latency (three runs, retrieval included). Quality: correctness judged by DeepSeek-chat (another model family) against reference answers, as in RAGAS [14], with 20% graded blind by hand; citation precision; abstention accuracy. **TCCA** = tokens per correct *and* correctly cited answer; **TPCA** ignores citations. The **exchange rate** = decode time per output token ÷ prefill time per input token. A variant is acceptable only if correctness drops by at most three points and abstention does not drop, a margin fixed in advance. Tests: McNemar and bootstrap intervals per question; regressions labelled by cause; τ optimism by stratified 5-fold cross-validation (20 repeats).

4. Results

Table 2. Results per variant over 40 questions (tokens and latency: means over three runs; quality: first run). Cited = citation precision; Abstain = abstention accuracy; TCCA = tokens per correct cited answer; TPCA = tokens per correct answer.

Variant	Input tokens	Output tokens	TTFT (s)	Latency (s)	Correct (%)	Cited (%)	Abstain (%)	TCCA	TPCA
V0 Baseline	6,487	133	22.20	26.12	88	0	71	53,012	7,573
V1 Adaptive retrieval	599	71	2.23	4.04	92	100	100	1,569	721
V2 Output budget	6,556	59	22.15	23.79	70	14	43	44,079	9,446
V3 Optimised	658	36	2.35	3.19	78	88	100	957	896
VO Oracle	368	33	1.21	1.98	82	100	100	486	486

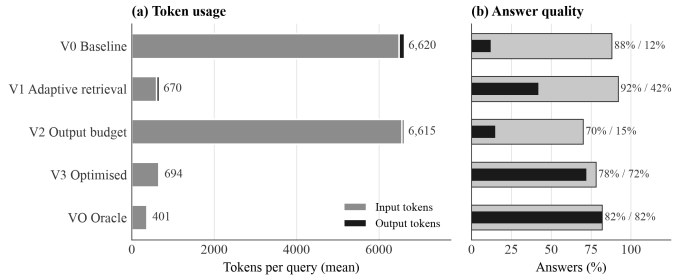


Figure 1. (a) Mean tokens per query: input (grey) and output (black). (b) Correct answers (light bar) and answers that are also correctly cited (dark bar).

Tokens and quality. T1 (V1) cut input tokens by 91% and latency from 26.1 s to 4.0 s, while correctness changed by +5.0 points (95% CI −10.0 to 17.5; McNemar p = 0.73): within the margin. T4 (V2) cut output tokens by 56% but correctness fell by 17.5 points (95% CI −32.5 to −5.0; McNemar p = 0.04). V3 combined both (90% fewer input and 73% fewer output tokens, 3.2 s), but its correctness fell by 10.0 points (95% CI −25.0 to 5.0; McNemar p = 0.34), missing the margin. Of the 7 questions V0 answered correctly and V3 did not, 6 had the right evidence in context but the short answer omitted a required detail. With T1, abstention accuracy rose from 71% to 100%; in V3, 15% of questions got "not found" without any LLM call. The baseline almost never cited a passage,

which inflates its TCCA; ignoring citations, TPCA still fell from 7,573 to 721 (V1) and 896 (V3).

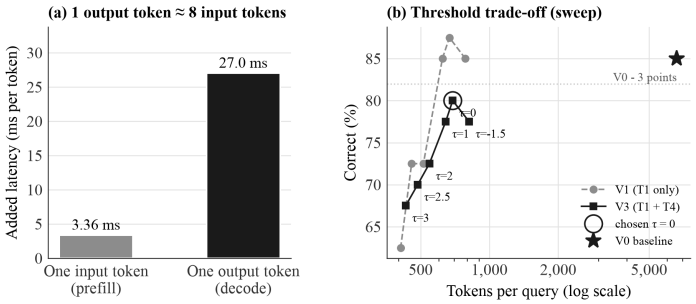


Figure 2. (a) Measured latency per input token (prefill) and per output token (decode). (b) Tokens per query against correctness for every swept τ (sweep run); the dotted line is the non-inferiority margin.

Latency and hardware. Prefill ran at 297 tokens/s and decode at 37 tokens/s, so one output token cost as much latency as 8 input tokens. Decode reached 65% of the memory-bandwidth ceiling of 57 tokens/s (120 GB/s ÷ 2.1 GB of weights), as expected for a bandwidth-bound phase. Long prompts slowed every output token (29.0 vs 24.3 ms), because each decode step also reads the prompt's KV cache (36 KiB per token).

Robustness. Cross-validating τ lowered V3 correctness from 80% to 73% (V1: 85% to 82%), so tuning on the test set is optimistic by up to 7 points. The judge agreed with the blind manual grades on 31 of 33 answers (Cohen's κ = 0.84); both disagreements were multi-part answers missing a step.

5. Discussion

RQ1. Input tokens can be cut by an order of magnitude without losing correctness: dropping low-scoring passages helped rather than hurt, in line with [4, 5]. Output is harder: the fixed three-sentence contract left out secondary facts, so T4 failed the margin; the oracle, which uses the same contract, scored only 82%, pointing to the contract, not retrieval, as the limit. **RQ2.** The two token types cost differently, but the ratio depends on the hardware: on this laptop GPU prefill is slow, so the baseline prompt dominated latency (first token after 22.2 of 26.1 s) and T1 saved far more time than T4. Where prefill is much faster relative to decode, as on data-centre accelerators [1, 2], output reduction gains weight, so the exchange rate should be measured per deployment. **RQ3.** Yes: abstention accuracy rose from 71% to 100%. At 100,000 queries a day, V3's measured savings would amount to about 583 million input tokens and 637 machine-hours per day on this hardware (a projection from measured means; about 187 USD at DeepSeek-Flash peak list prices).

6. Conclusion

(1) Adaptive retrieval is the clear win. V1 cut input tokens from 6,487 to 599 (91%) and latency from 26.1 s to 4.0 s (6.5 times faster), while correctness went from 87.5% to 92.5% ($p = 0.73$) and abstention from 71% to 100%. **(2) A fixed output budget is not safe.** It cut output tokens by 56% but correctness fell 17.5 points ($p = 0.039$, the only significant change), and by 10.0 points when combined with T1. **(3) On this hardware the input side dominates:** one output token costs 8 input tokens of latency ($R^2 = 0.97$), and decode already runs at 65% of its memory-bandwidth limit. **(4) The evidence holds up:** the judge matches blind manual grading ($\kappa = 0.84$), but tuning τ on the test set overstates V3 by 7 points. **Recommendation:** use adaptive retrieval by default with τ set on held-out questions, and replace the fixed output budget with one that adapts to the question. **Limitations:** 40 questions from one domain; one 3B model on one fanless laptop; a judge slightly lenient on multi-part answers. **Future work:** question-adaptive output budgets, history compaction, a citation-gated cascade, and the exchange rate on a server GPU.

Use of AI tools. Claude Code (Anthropic) was used to speed up coding of the RAG pipeline and to help generate scripts (evaluation, figures and report build), and to set up Ollama. The architecture and framework, the literature research on effective token-reduction methods, the choice of what to test, the test questions, the blind manual grading of answers and the analysis of the results are my own work.

References

[1] P. Patel *et al.*, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proc. ISCA*, 2024. arXiv:2311.18677.

[2] Y. Zhong *et al.*, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proc. OSDI*, 2024. arXiv:2401.09670.

[3] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023. arXiv:2309.06180.

[4] N. F. Liu *et al.*, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

[5] F. Shi *et al.*, “Large language models can be easily distracted by irrelevant context,” in *Proc. ICML*, 2023. arXiv:2302.00093.

[6] Anthropic, “Effective context engineering for AI agents,” Anthropic Engineering Blog, 2025. [Online]. Available: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[7] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in *Proc. NAACL*, 2024. arXiv:2403.14403.

[8] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *Proc. ICLR*, 2024. arXiv:2310.11511.

[9] F. Xu, W. Shi, and E. Choi, “RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation,” in *Proc. ICLR*, 2024. arXiv:2310.04408.

[10] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” in *Proc. EMNLP*, 2023. arXiv:2310.05736.

[11] S. Xu, W. Xie, L. Zhao, and P. He, “Chain of draft: Thinking faster by writing less,” arXiv:2502.18600, 2025.

[12] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” arXiv:2305.05176, 2023.

[13] Y. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” arXiv:2312.10997, 2023.

[14] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in *Proc. EACL (System Demonstrations)*, 2024. arXiv:2309.15217.

[15] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.

[16] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019. arXiv:1908.10084.

Appendix A. Experimental Record

Table 3. Everything that was built, run and checked for this study. All figures are counted from the files in the repository (results/ and data/).

Step	What was done	Size and outcome
Knowledge base	Kubernetes troubleshooting documentation downloaded, cleaned and split; dated copy stored	111 sections, 232 passages, ~36,000 tokens
Test set	Questions and reference answers written by hand; gold passages checked manually	40 questions (25 single, 8 two-part, 7 unanswerable)
Threshold sweep	Six τ values for V1 and V3, plus V0 as reference	520 runs; $\tau = 0$ chosen
Main experiment	5 variants × 40 questions × 3 repeats, interleaved per question, resumable	600 runs
Local inference	Qwen2.5-3B (Q4_K_M) in Ollama, temperature 0, prompt-prefix reuse prevented	975 LLM calls; 2.28 M input and 68,442 output tokens; 2.8 h of timed answers
Automatic grading	DeepSeek-chat judge on every distinct answer; rule for "not found" answers	293 answers graded (267 by judge, 26 by rule)
Manual grading	Random 20% of judge-graded main-run answers, graded blind by hand	33 answers; 31 agree; $\kappa = 0.84$
Latency model	Regression of LLM time on input and output tokens	$n = 543$, $R^2 = 0.97$; 3.36 vs 27.0 ms per token
Hardware check	Decode speed against the memory-bandwidth ceiling; KV cache size from model metadata	37 of 57 tokens/s; 36 KiB per token
Significance tests	Exact McNemar test and bootstrap 95% CI for each variant against V0	4 comparisons; only V2 significant ($p = 0.039$)
τ robustness	Stratified 5-fold cross-validation, repeated 20 times	optimism 7.2 points (V3), 2.6 points (V1)
Error analysis	Every answer worse than the baseline, labelled automatically by cause	18 regressions (V1: 3, V2: 8, V3: 7)
Scale projection	V3 savings at 100,000 queries per day	583 M tokens, 637 machine-hours, ~187 USD
Software checks	Offline end-to-end test with fake models, run automatically on Python 3.11 to 3.13	all checks pass